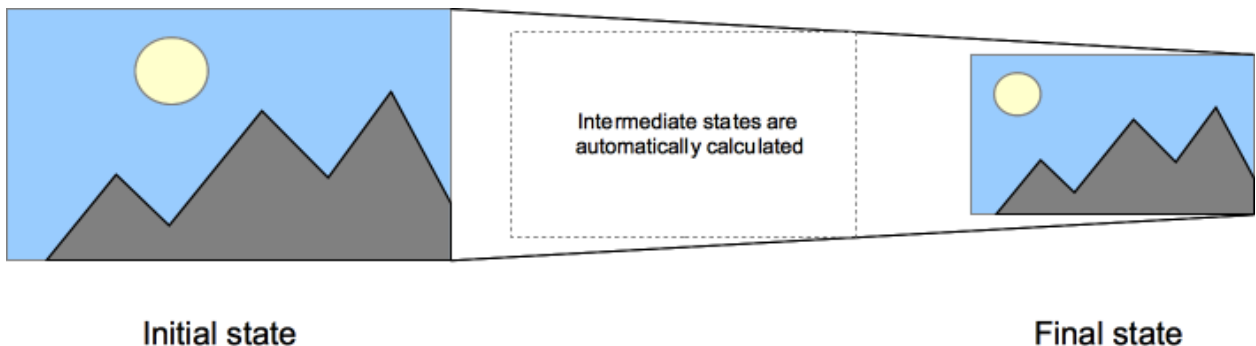


CSS Transitions

Normally when the value of a CSS property changes, the rendered result is instantly updated, with the affected elements immediately changing from the old property value to the new property value. This section describes a way to specify transitions using new CSS properties. These properties are used to animate smoothly from the old state to the new state over time.



Note: Transition will be started when you move cursor on it. You must need to specify time duration otherwise Transition have no effect. The default time value is zero.

You need to specify two things for create CSS transition.

- The CSS property on which you want to add an effect.
- The time duration of the effect.

Transitions support only pixels based and colour based properties only.

Ex: width, height, opacity, font-size, border-width, background-colour, colour, border-colour etc.

selector{

property : old-value;

property : old-value;

...

transition: **property** time (sec/ms), **property** time (sec/ms), ...

}

Selector:hover{

property :new-value;

property : new-value;

....

}

Property	Description
transition	A shorthand property for setting the four transition properties into a single property
transition-delay	Specifies when the transition effect will start.
transition-duration	Specifies how many seconds or milliseconds a transition effect takes to complete
transition-property	Specifies the name of the CSS property the transition effect is for.
transition-timing-function	Specifies the speed curve of the transition effect. • ease specifies a transition effect with a slow start, then fast, then end slowly (this is default) • linear specifies a transition effect with the same speed from start to end • ease-in specifies a transition effect with a slow start • ease-out specifies a transition effect with a slow end • ease-in-out specifies a transition effect with a slow start and end

Example1

```

<!DOCTYPE html>
<html>
<head>
<style>
img {
background: red;
transition: width 2s;
}
img:hover {
width: 500px;
}
</style>

```

```
</head>
<body>
<h1>The transition Property</h1>
<imgsrc="car1.jpg" width="100" height="100">
<p>Hover over the img element, to see the transition effect</p>
</body>
</html>
```

Example2

```
<!DOCTYPE html>
<html>
<head>
<style>
    .d1 {
        width: 100px;
        height: 100px;
        background: #ff33cc;
        transition: width 500ms;
    }
    .d1:hover {
        width: 500px;
    }
    .d2 {
        width: 100px;
        height: 100px;
        background: red;
        transition: height 2s;
    }
    .d2:hover {
        height: 300px;
    }
</style>
</head>
<body>
    <h1>The transition Property</h1>
    <p>get mouse over the div element, to see the transition width effect</p>
    <div class="d1"></div>

    <br><br>

    <p>get mouse over the div element, to see the transition height effect</p>
    <div class="d2"></div>
</body>
</html>
```

Example3

```
<!DOCTYPE html>
<html>
<head>
<style>
    div {
        width: 100px;
        height: 100px;
        background: #ff3300;
position: relative;
        color: #00ff33;
        font-size: 25px;
        transition: background-color 2s, font-size 2s, width 2s;
    }
    div:hover {
        background-color: blue;
        font-size: 100px;
        width: 400px;
    }
</style>
</head>
<body>
    <h1>The transition with mutiple Properties</h1>

    <div>See effect</div>
</body>
</html>
```

CSS Transforms

Transforms are used to display the element in a different visual dimension.
Using predefine functions

Types of Transforms:

- `matrix(1, 2, 3, 4, 5, 6...);`
- `translate(x, y)` `translateX(x)` `translateY(y)`
 `translateZ(z)`
- `scale(x,y);` `scaleX(x)` `scaleY(y)` `scaleZ(z)`
- `rotate(deg)` `rotateX(x-deg)` `rotateY(y-deg)`

□ skew(Xdeg, Ydeg) skewX(Xdeg) skewY(Ydeg)

transform: fun1(..) fun2(..) fun3(..) ...;

```
/* Keyword values */
transform: none;

/* Function values */
transform: matrix(1.0, 2.0, 3.0, 4.0, 5.0, 6.0);
transform: matrix3d(1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1);
transform: perspective(17px);
transform: rotate(0.5turn);
transform: rotate3d(1, 2.0, 3.0, 10deg);
transform: rotateX(10deg);
transform: rotateY(10deg);
transform: rotateZ(10deg);
transform: translate(12px, 50%);
transform: translate3d(12px, 50%, 3em);
transform: translateX(2em);
transform: translateY(3in);
transform: translateZ(2px);
transform: scale(2, 0.5);
transform: scale3d(2.5, 1.2, 0.3);
transform: scaleX(2);
transform: scaleY(0.5);
transform: scaleZ(0.3);
transform: skew(30deg, 20deg);
transform: skewX(30deg);
transform: skewY(1.07rad);

/* Multiple function values */
transform: translateX(10px) rotate(10deg) translateY(5px);
transform: translate(10px, 0, 20px) rotateY(3deg);
```

Example1

```
<!DOCTYPE html>
<html>
<head>
<style>
  img {
```

```

border: solid red;
transform: rotate(50deg);
width: 140px;
height: 100px;
}

div {
border: solid red;
transform: translate(30px, 20px) rotate(20deg);
width: 140px;
height: 60px;
}
</style>
</head>
<body>
  <h1>The transition with mutiple Properties</h1>

  <div>See effect</div><br>
  <imgsrc="car3.png">

</body>
</html>

```

Example2

```

<!DOCTYPE html>
<html>
<head>
<style>
  div {
width: 200px;
height: 200px;
background-color:#00ccff;
margin-left: 150px;
box-shadow: 10px 10px10px10px #ff0000;
  }
  div:hover{
background-color:#0099ff;
transform: translate(30px, 30px); /* scale(1.5); */
  }
</style>
</head>
<body>
  <h1>The transform Properties</h1>
  <div>See effect</div>

</body>
</html>

```

Example3

```
<!DOCTYPE html>
<html>
<head>
<style>
    div {
        width: 200px;
        height: 200px;
        background-color:#6633ff;
        margin-left: 100px;
        box-shadow: 5px 5px5px5px #ff0000;
        transition: transform 1s;
    }
    div:hover{
        background-color:#0099ff;
        transform: rotate(90deg) scale(1.5);
    }
</style>
</head>
<body>
    <h1>Transform with Transition Properties</h1>
    <div>See effect</div>
</body>
</html>
```

Animations

- Animations are “group of transitions/transform/properties”, which will be performed one after another.
- Transition contains two points only (starting point, ending point).
- Animation contains multiple points of milestones.

Syn:

@keyframes Ani-name

```
{  
  from{ prop:value; prop:value; prop:value;...} old status  
  to{prop:new-value; prop:new-value; prop:new-value;...} new status  
}
```

@keyframesAni-name

```
{  
  0%{ prop:value; prop:value; prop:value;...}  
  25%{ prop:value; prop:value; prop:value;...}  
  33%{ prop:value; prop:value; prop:value;...}  
  75%{ prop:value; prop:value; prop:value;...}  
  100%{ prop:value; prop:value; prop:value;...}  
}
```

Mapping to tags:

Selector{

animation-name:ani-name;

animation-duration: Ns;

animation-direction: normal/reverse/alter;

animation-delay:Ns;

animation-iteration-count:count/infinite; default is 1

}